502094 - Software Deployment, Operations And Maintenance

Module 2: Linux Operations, Shell Scripting & Git Essentials

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Module Overview

- Build **foundational Linux skills** for backend and DevOps engineers
- Understand **how the operating system is structured**
- Learn **why** each command is needed in real deployment scenarios
- Practice **example-based learning**: every concept has a concrete demo
- Topics in this module:
  - System navigation & file operations
  - Text processing & log analysis
  - Process & resource management
  - Input/output file handling
  - System administration & security
  - Software installation with `apt`

Why Linux Matters for Backend & DevOps

- Most production **web servers**, containers, and cloud VMs run Linux
- Tools like **Docker, Kubernetes, Nginx, Node.js, Java** are deployed on Linux
- Common daily tasks:
  - Deploying new versions of web applications
  - Inspecting logs when an incident happens
  - Restarting or debugging failing services
- Strong Linux foundations help you:
  - Communicate effectively with SRE/DevOps teams
  - Troubleshoot issues under time pressure
  - Automate repetitive operational work

Learning Roadmap

1. System Navigation & File Operations
2. Text Processing & Log Inspection
3. Process & Resource Management
4. Input & Output File Management
5. System Administration & Security Basics
6. Software Installation & Package Management
7. Shell Scripting Fundamentals

System Navigation & File Operations

The Linux Filesystem Model

- Linux uses a single **tree** starting from the root: `/`
- Everything is a **file** (including devices and sockets)
- Paths:
  - **Absolute path**: starts from `/` (e.g. `/home/student/projects`)
  - **Relative path**: based on current directory (e.g. `../logs`)
- Essential idea:
  - You must always know **where you are** and **where your app lives**
  - Use `pwd` (print working directory) to confirm the current location

Important System Directories

- `/etc` – configuration files for services (e.g. `nginx`, `ssh`)
- `/var` – variable data (logs, queues, caches)
- `/usr` – user-level programs and libraries
- `/opt` – optional, third-party applications
- `/home` – home directories of normal users
- `/srv` – data for services (web apps, databases, etc.)
- As a backend developer you typically:
  - Edit configs in `/etc`
  - Read logs in `/var/log`
  - Deploy apps under `/srv` or `/var/www`

Navigating the Filesystem (`pwd`, `cd`, `ls`)

- `pwd` – show current directory
- `cd` – change directory
  - `cd /var/log` – go to logs directory
  - `cd ..` – go one level up
  - `cd ~` – go to home
- `ls` – list directory content
  - `ls` – names only
  - `ls -l` – long format (permissions, owner, size)
  - `ls -la` – include hidden files
- These are the **fundamental tools** you use every time you log in to a server

Why Copy, Move, and Remove Files?

- In real projects you often need to:
  - Backup configuration files before editing
  - Move build artifacts to deployment directories
  - Clean up old log archives
- Core commands:
  - `cp` – copy files or directories
  - `mv` – move or rename
  - `rm` – remove files and directories
- Example scenario:
  - Backup Nginx config before editing and reloading the service

`cp`, `mv`, `rm` in Practice

- Copy a config file before changes:

```
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
```

- Move or rename a log:

```
mv app.log app.log.old
```

- Remove temporary build directory:

```
rm -r build_temp/
```

- Safety tip:
  - Prefer `rm -ri` on servers so you confirm before deleting recursively

Why We Need `find`

- Real servers contain **thousands of files**
- You may not remember **exact locations** of configs or logs
- `find` helps you search:
  - By name
  - By file type
  - Within specific directories
- Typical backend tasks:
  - Locate all log files for a specific service
  - Find all `.env` files to review secrets handling

Using `find` Effectively

- Search by name in `/var/log`:

```
sudo find /var/log -name "nginx*"
```

- Search for all `.log` files in current directory tree:

```
find . -name "*.log"
```

- Search for directories used for deployments:

```
find /srv -type d -name "release*"
```

- You can even delete unwanted files:

```
find . -name "*.tmp" -delete
```

Why File Permissions Matter

- Servers often host **multiple services** and **multiple users**
- Permissions protect:
  - Application source code
  - Configuration files containing secrets
  - Log files with sensitive information
- You must understand:
  - Who **owns** the file
  - Which operations (read, write, execute) are allowed
- Misconfigured permissions can:
  - Break deployments
  - Expose confidential data

Understanding `ls -l` Output

- Example:

```
ls -l app.log
-rw-r----- 1 www-data www-data 4096 Oct 18 10:00 app.log
```

- `-rw-r-----` breaks into:
  - File type: `-` (regular file)
  - User: `rw-` (read, write). Group: `r--` (read only). Others: `---` (no access)
- Owner/group:
  - User: `www-data`
  - Group: `www-data`
- As a DevOps engineer you decide which users and groups can read or edit which files

Changing Permissions and Ownership

- `chmod` – change permissions:

```
chmod u+x deploy.sh        # allow user to execute
chmod 640 app.log          # rw- r-- ---
```

- `chown` – change owner and group:

```
sudo chown www-data:www-data app.log
```

- Typical patterns:
  - Application files owned by deployment user
  - Logs owned by service user (`www-data`, `nginx`, etc.)
  - Only administrators can modify config files in `/etc`

Compression & Archiving: Why Use `zip`?

- Production servers generate **large log files**
- You often need to:
  - Archive logs before deletion
  - Download a compressed copy for analysis
  - Send small bundles of files to teammates
- `zip` and `unzip` provide:
  - Easy compression
  - Cross-platform compatibility
- First ensure packages are installed using `apt`

Using `zip` and `unzip`

- Install tools (if needed):

```
sudo apt update
sudo apt install zip unzip
```

- Archive a logs directory:

```
zip -r logs-2025-10-18.zip /var/log/myapp/
```

- Extract archive to `/tmp`:

```
unzip logs-2025-10-18.zip -d /tmp/myapp-logs/
```

- Use case:
  - Compress logs, download them to your laptop, and analyze locally

Text Processing & Log Inspection

Why Logs Are Critical

- When a system fails, logs tell you **what happened and when**
- Logs help to:
  - Debug errors (HTTP 500, database errors)
  - Investigate security incidents
  - Monitor user behavior and performance issues
- Common log locations:
  - `/var/log/syslog`
  - `/var/log/auth.log`
  - `/var/log/nginx/access.log`
  - Application-specific log directories under `/var/log/`

Viewing Large Files Safely

- `cat` – print entire file (good for small files)
- `less` – scroll through large files:

```
less /var/log/syslog
```

  - Move with arrow keys, PageUp/PageDown
  - Search inside `less` with `/pattern`. Quit with `q`

- `head` / `tail`:

```
head -n 30 app.log   # first 30 lines
tail -n 30 app.log   # last 30 lines
```

- These tools prevent your terminal from being flooded with output

Real-Time Log Monitoring with `tail -f`

- During deployment you want to see **live logs**
- `tail -f` follows the end of a file:

```
tail -f /var/log/nginx/access.log
```

- Use cases:
  - Watch incoming requests while testing new endpoints
  - Observe errors immediately after restarting a service
- Combine with `grep` to filter only errors:

```
tail -f access.log | grep " 500 "
```

Searching Logs with `grep`

- `grep` finds lines matching a pattern:

```
grep "ERROR" app.log
```

- Useful options:
  - `-i`: case insensitive, `-n`: show line number
  - `-r`: recursive search in directories
- Examples:

```
grep -n "Exception" /var/log/myapp/error.log
grep -ri "database connection" /var/log/myapp/
```

- This is your **primary tool** for quickly locating relevant log entries

Counting and Pipelines: `wc` + `grep`

- `wc -l` counts lines:

```
wc -l access.log
```

- Pipelines connect commands with `|`:

```
grep " 500 " access.log | wc -l
```

- Example use cases:
  - Count number of HTTP 500 errors
  - Count failed SSH login attempts:

```
grep "Failed password" /var/log/auth.log | wc -l
```

- Thinking in pipelines is key to powerful **Linux one-liners**

Process & Resource Management

Understanding Processes

- A **process** is a running program instance
- Each process has:
  - PID (Process ID)
  - Parent process
  - User and group owner
  - Resource usage (CPU, memory, I/O)
- As a backend or DevOps engineer, you must know:
  - Which processes are running your application
  - How much resource they consume
  - How to stop or restart them safely

Inspecting Processes: `ps`, `top`, `htop`

- `ps` – snapshot of current processes:

```
ps aux | grep nginx
```

- `top` – real-time system monitor:

```
top
```

  - Sort by CPU or memory usage

- `htop` – more user-friendly process viewer:

```
sudo apt install htop
htop
```

- These tools help you understand **what is consuming system resources**

Managing Processes and Signals

- Graceful stop:

```
kill 1234          # sends SIGTERM by default
```

- Force stop:

```
kill -9 1234       # sends SIGKILL
```

- Keyboard shortcuts:
  - `Ctrl + C` – send SIGINT to foreground process
- Best practices:
  - Prefer **SIGTERM** (give app chance to clean up)
  - Use **SIGKILL** only when the process is stuck

Monitoring CPU, RAM, I/O, and Network

- CPU and memory:
  - `top` / `htop` show system-wide usage
- Disk I/O:
  - `iostat` (in `sysstat` package) for low-level I/O statistics
- Network:
  - `ss -tulpn` to see listening ports and services
- Typical workflow when users report slowness:
  i. Check CPU and RAM usage
  ii. Check disk I/O saturation
  iii. Check network connections and open ports

Input & Output File Management

Creating and Editing Files (`touch`, `nano`)

- Create an empty file:

```
touch README.txt
```

- Edit files in terminal with `nano`:

```
nano /etc/nginx/sites-available/default
```

- Use cases:
  - Create environment variable files
  - Edit service configuration before reload
- Basic `nano` keys:
  - `Ctrl + O` – write file
  - `Ctrl + X` – exit editor

Redirecting Output to Files

- Write to a file using `>`:

```
echo "APP_ENV=production" > .env
```

- Append using `>>`:

```
echo "LOG_LEVEL=info" >> .env
```

- Redirect errors using `2>`:

```
some_command 2> errors.log
```

- Real-world pattern:
  - Generate config fragments or environment files during deployment scripts

Links and Release Management (`ln`)

- Linux allows **multiple names** for the same file or directory
- `ln` creates a **hard link**
- `ln -s` creates a **symbolic link (symlink)**`:

```
ln -s /srv/myapp/releases/v1 /srv/myapp/current
```

- Why symlinks are useful:
  - Switch between releases by updating `current`
  - Web server points to `/srv/myapp/current/public`
  - Rollback is as simple as changing the symlink

System Administration & Security Basics

Users, Groups, and Security Model

- Every process runs as a particular **user**
- Users belong to **groups**
- Access to files and devices is controlled by:
  - User ID
  - Group ID
  - Permission bits (read, write, execute)
- Principle of least privilege:
  - Do not run web apps as `root`
  - Use dedicated service accounts (`www-data`, `appuser`)

Managing Users and Groups

- Create a new user with a home directory:

```
sudo useradd -m -s /bin/bash devuser
sudo passwd devuser
```

- Add user to a group (e.g. `docker`):

```
sudo usermod -aG docker devuser
```

- Check user groups:

```
groups devuser
```

- For DevOps work:
  - Create controlled users for CI/CD deployments. Avoid sharing `root` credentials

Sudo Privileges and SSH Hardening

- Grant administrative access using `sudo`:

```
sudo usermod -aG sudo devuser
```

- SSH best practices:
  - Use **public key authentication** instead of passwords
  - Disable root login: `PermitRootLogin no` in `/etc/ssh/sshd_config`
  - Optionally disable password logins when keys are configured
- Always keep one working SSH session open when modifying SSH settings

Security Logs and Basic Protection

- Important security-related logs:
  - `/var/log/auth.log` – authentication events
  - `/var/log/syslog` – general system events
- Tools like **Fail2ban**:
  - Monitor log files for repeated failures
  - Add firewall rules to block suspicious IP addresses
- Daily routine for a secure server:
  - Review `auth.log` for unusual login attempts
  - Ensure Fail2ban or similar protection is active

Software Installation & Package Management

Package Management with `apt`

- Linux distributions provide **package managers** to install software
- On Debian/Ubuntu, we use `apt`:
  - Resolves dependencies automatically
  - Downloads from trusted repositories
- Configuration files:
  - `/etc/apt/sources.list`
  - `/etc/apt/sources.list.d/*.list`
- Practical tasks:
  - Install tools like `htop`, `zip`, `nginx`
  - Keep servers updated with security patches

Common `apt` Commands

- Refresh package index:

```
sudo apt update # or run after adding new repositories
```

- Upgrade installed packages:

```
sudo apt upgrade
```

- Install new tools:

```
sudo apt install zip unzip htop
```

- Remove a package:

```
sudo apt remove unzip
```

Shell Scripting (Bash/Zsh) Foundations

Why Learn Shell Scripting?

- Many operational tasks are **repetitive**:
  - Building and deploying applications
  - Restarting services in correct order
  - Backing up databases and logs
- Shell scripts allow you to:
  - Combine Linux commands into **reusable programs**
  - Automate manual runbooks
  - Reduce human errors during deployments
- Bash and Zsh are the most common shells on Linux servers

First Bash Script Example

- Create a file:

```
nano hello.sh
```

- Add content:

```
#!/usr/bin/env bash

echo "Hello from Module 2!"
```

- Make script executable and run:

```
chmod +x hello.sh
./hello.sh
```

- This is the building block for more advanced automation scripts

Shell Function: Compile, Run, and Clean C Program

- Define a reusable function in a script or in `~/.bashrc`:

```
build_run_c() {
  clear
  local src="$1"

  if [ -z "$src" ]; then
    echo "Usage: build_run_c main.c"
    return 1
  fi

  gcc "$src" -o a.out && ./a.out
  rm -f a.out
}
```

Shell Function: Compile, Run, and Clean C Program

- Usage:

```
build_run_c main.c
```

- This function:
  - Clears the screen
  - Compiles the C source file
  - Runs the program
  - Removes the executable after execution

Git and Github

Section Overview

- Understand the role of **version control** in modern software development
- Learn core **Git concepts** and how they map to your daily workflow
- Connect local Git repositories with **GitHub** as a remote hosting platform
- Practice typical scenarios:
  - Saving work with commits
  - Pushing to GitHub
  - Navigating history and branches
  - Undoing and cleaning changes safely
  - Handling conflicts and collaborating in teams

Why Version Control?

- Real projects evolve through **many changes** over time
- Without version control:
  - Files like `final_v3_really_final.zip` appear
  - Hard to track who changed what and when
  - Difficult to roll back when something breaks
- With **Git**:
  - You have a complete **history** of the project
  - You can safely experiment on **branches**
  - Teams can collaborate without overwriting each other

Introduction to Git & GitHub

What Is Git?

- **Git** is a **distributed version control system**
- Key ideas:
  - Every developer has a **full copy** of the repository and its history
  - Changes are recorded as **commits** (snapshots with messages)
  - You can move between commits, compare versions, and restore files
- Designed for:
  - Speed
  - Non-linear development with **branches**
  - Collaboration in small or large teams

What Is GitHub?

- **GitHub** is a **cloud platform** that hosts Git repositories
- Adds collaboration features on top of Git:
  - Remote repositories
  - Pull requests and code reviews
  - Issues, discussions, and project tracking
- For our course:
  - Git runs **locally** on your machine
  - GitHub acts as the **remote backup and collaboration hub**

Local vs Remote Repositories

- **Local repository**:
  - Lives on your machine (e.g. in `~/projects/my-app`)
  - You run commands like `git status`, `git add`, `git commit`
- **Remote repository** (e.g. on GitHub):
  - Central place where everyone pushes and pulls changes
  - Accessed via HTTPS or SSH
- Typical flow:
  - Work locally → commit → **push** to GitHub
  - Get others' work from GitHub via **pull**

Installing Git & Creating a GitHub Account

Installing Git

- On Ubuntu / Debian:

```
sudo apt update
sudo apt install git
```

- Check version:

```
git --version
```

- On Windows / macOS:
  - Use official installers from the Git website or package managers
  - Ensure `git` works from your terminal or PowerShell

Creating a GitHub Account

- Go to the GitHub website and sign up with your email
- Choose a **username** that you can reuse professionally
- Configure at least:
  - Display name
  - Avatar
  - Public email (optional)
- Later we will connect your local Git to this GitHub account

GitHub CLI (`gh`) Overview

- **GitHub CLI** (`gh`) is a command-line tool to interact with GitHub
- Useful for:
  - Authenticating from the terminal
  - Creating repositories and pull requests
  - Managing issues and releases
- Installation on Ubuntu:

```
# the command is pretty long, so refer to official docs
```

Logging In to GitHub from the CLI

Authenticating with `gh auth login`

- Use GitHub CLI to log in from your terminal:

```
gh auth login
```

- Typical steps:
  - Choose GitHub.com
  - Choose HTTPS as the protocol
  - Follow the browser-based authentication flow
- After login:

```
gh auth status
```

to verify you are authenticated

Authenticating Git Over HTTPS

- Git itself can use **personal access tokens (PATs)** as HTTPS passwords
- When pushing via HTTPS:
  - Git will prompt for username and password
  - Use your GitHub **username** and **token** instead of your normal password
- Many workflows now prefer:
  - Logging in via `gh`
  - Or using SSH keys for long-term use

Basic Git Configuration & Commands

Configuring Your Identity

- Set your name and email globally (once per machine):

```
git config --global user.name "Mai Van Manh"
git config --global user.email "maivanmanh@tdtu.edu.vn"
```

- Check your configuration:

```
git config --global --list
```

- This information is stored in each commit you create

Creating a New Repository

- Initialize Git in an existing project:

```
cd my-project
git init
```

- Or clone an existing repo from GitHub:

```
git clone https://github.com/username/my-project.git
```

- After initialization:
  - Git starts tracking changes in that folder
  - You can use `git status` to see what is happening

Understanding Git States

- Three main areas:
  - **Working directory** – your files on disk
  - **Staging area** – files prepared for the next commit
  - **Repository (history)** – stored commits
- Typical cycle:

```
edit files → git add → git commit
```

- Command to inspect status:

```
git status
```

Saving Work with Commits & Pushing to GitHub

Creating Commits

- Stage changes:

```
git add main.c
git add src/ utils/
```

- Create a commit with a message:

```
git commit -m "Implement basic HTTP handler"
```

- Good commit messages:
  - Short, imperative: “Add health-check endpoint”
  - Describe **why** and **what** changed

Connecting to a Remote and Pushing

- Add a remote repository (GitHub):

```
git remote add origin https://github.com/username/my-project.git
```

- Push the current branch to GitHub:

```
git push -u origin main
```

- After first push:
  - Next times, just run `git push` on the same branch
- Verify on GitHub:
  - The code and commit history should appear in the web UI

Viewing History and Moving in Time

- View commit history:

```
git log
```

- More compact view:

```
git log --oneline --graph --all
```

- Check out a past commit (detached HEAD):

```
git checkout <commit-id>
```

- This is useful for:
  - Investigating when a bug was introduced
  - Comparing current behavior with an older version

Working with Branches

Why Branches?

- Branches let you **develop features in isolation**
- You can:
  - Experiment without breaking the `main` branch
  - Review and test changes before merging
- Typical pattern:
  - Create a branch for each task or feature
  - Merge back into `main` after review

Creating and Switching Branches

- Create a new branch from current commit:

```
git branch feature/login
```

- Switch to a branch:

```
git checkout feature/login
```

- Create and switch in a single command:

```
git checkout -b feature/login
```

- Always confirm with:

```
git branch
```

- The current branch is marked with `*`.

Merging Branches and Deleting Them

- Merge a completed feature branch into `main`:

```
git checkout main
git merge feature/login
```

- After successful merge, delete the branch:

```
git branch -d feature/login
```

- Push branch to GitHub for review:

```
git push -u origin feature/login
```

- On GitHub, you typically open a **pull request** for review before merging

Undoing Changes & Cleaning the Working Tree

Returning to the Latest Commit

- Sometimes you want to **discard local changes**:
  - Files modified but not staged
  - Untracked files created during experiments
- To restore tracked files to the last commit:

```
git restore path/to/file.c
```

- To restore everything tracked:

```
git restore .
```

Cleaning Untracked Files and Folders

- Remove untracked files (be careful):

```
git clean -f
```

- Remove untracked directories as well:

```
git clean -fd
```

- Typical use case:
  - Return working tree to **exactly** the last commit
  - Remove temporary build artifacts and generated files

Resetting to a Commit (Advanced)

- Move the current branch pointer to a specific commit:

```
git reset --hard <commit-id>
```

- Effects:
  - Working directory is updated to match that commit
  - Commits after that point are removed from the current branch
- Use with caution:
  - Avoid rewriting history that has already been **pushed to others**

Handling Detached HEAD and Mistakes

Detached HEAD Explained

- When you run:

  ```
  git checkout <commit-id>
  ```

  Git enters **detached HEAD** state
- You are no longer on a branch:
  - Commits you create here are not attached to a named branch
- If you only want to **inspect** old code:
  - Do not commit in detached HEAD
  - Use it for temporary viewing or testing

Recovering from Edits in Detached HEAD

- Common mistake:
  - Check out an old commit to view code
  - Accidentally edit a file
  - Try to switch back to `main` and Git refuses due to uncommitted changes

Recovering from Edits in Detached HEAD

- Solutions:
  - If you do not need the changes:

    ```
    git restore .
    git checkout main
    ```

  - If you want to keep them:
    - Create a new branch from this state:

      ```
      git checkout -b experiment-from-old-commit
      ```

Working in Teams & Handling Conflicts

How Merge Conflicts Happen

- Scenario with 2 developers:
  - Both start from the same commit on `main`
  - Both modify the **same lines** in the same file
  - The second merge or pull will produce a **conflict**
- Git marks conflicts in the file:

```
<<<<<<< HEAD
your local changes
=======
changes from incoming commit
>>>>>>> other-branch
```

- Conflicts are **normal** in collaborative work

Resolving a Simple Conflict

- Steps to resolve:
  i. Open the conflicted file(s)
  ii. Decide which lines to keep (or combine both)
  iii. Remove the conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`)
  iv. Stage the resolved file:

```
git add conflicted-file.c
```

  v. Complete the merge:

```
git commit
```

- Communicate with your teammate to understand intent

Pulling Changes Safely

- Regular workflow when collaborating:

```
git checkout main
git pull origin main
```

- Before starting new work:
  - Ensure your local `main` is **up to date**
- If you have local changes:
  - Commit them first, or
  - Use `git stash` to temporarily save your work before pulling

Branch Protection, Pull Requests & Best Practices

Branch Protection on GitHub

- Teams often protect the `main` (or `master`) branch:
  - Prevent direct pushes
  - Require pull requests
  - Require status checks (tests) to pass
- On GitHub repository settings:
  - Configure **branch protection rules**
- Benefits:
  - Avoid accidental force pushes
  - Enforce code review and testing before merging

Pull Requests (PRs) Workflow

- Typical GitHub workflow:
  i. Create branch: `feature/login`
  ii. Commit changes locally
  iii. Push branch to GitHub:

```
git push -u origin feature/login
```

  iv. Open a **pull request** from `feature/login` into `main`
  v. Request reviews, address comments, and then merge
- Pull requests create a clear **history of changes** and discussions

Best Practices When Working with Git & GitHub

- Make small, focused commits with meaningful messages
- Pull updates from `main` regularly to reduce conflicts
- Avoid committing:
  - Secrets (passwords, API keys)
  - Build artifacts or large binary files unless necessary
- Use `.gitignore` to exclude temporary and generated files
- Never rewrite public history unless the team agrees

Summary & Next Steps

- You have learned:
  - Core concepts of **Git** and **GitHub**
  - How to configure your identity and create repositories
  - How to commit, push, branch, merge, and undo safely
  - How to handle detached HEAD, clean working trees, and conflicts
  - The basics of branch protection, pull requests, and good practices
- Next steps:
  - Practice each scenario on a sample repository
  - Use Git and GitHub throughout the course projects